STracer Deployment Workflow and Data Protection

Revision 2.0 | 30 October 2026 | Backend-specific operating document

1. Purpose and actual deployment scope

This document defines release preparation, staging validation, production rollout, encryption and credential handling, operational checks, monitoring and recovery for the STracer backend. It is based on the current repository and distinguishes implemented controls from infrastructure settings that require verification.

The backend is a C# ASP.NET Core application targeting .NET 8. Its existing deployment helper targets Windows Server/IIS and runs `SignalTracker.dll` through the ASP.NET Core Module. A Dockerfile provides a separate container packaging option; container orchestration and automated container rollout are not established by this repository.

The React frontend, Android app and external Python analytics have separate delivery responsibilities. Their exact build commands, source revisions and hosting details must be supplied by their owners. The backend repository alone does not prove that those components have been built or deployed.

Assign one release identifier to the coordinated release and record the frontend/backend commits and individual artifact hashes. Example: `stracer-20261030-01`. This format also satisfies the backend packaging script's letters/digits/hyphens-only release-name validation.

Component	Actual backend relationship
Web portal	HTTPS APIs; email/password login; protected cookies and company/role-scoped access.
Android/mobile collection	Existing mobile APIs and RF upload APIs. The default enrollment path requires device approval. An optional signed trusted-login path now supports automatic authorization, but is disabled until the external login backend and app implement the verification/signing contract.
Application host	Windows IIS is supported by the existing deployment script; the repository web.config uses in-process hosting.
India database	MySQL selected through `ConnectionStrings:MySqlConnection`.
Taiwan database	Separate MySQL connection selected through `ConnectionStrings:MySqlConnection2`.
Redis	Cache and supporting login-state operations; current portal validation also relies on the authoritative regional database login-lock service.
RF object storage	Ace Cloud S3-compatible storage, production bucket `dcra-bucket`, endpoint `https://s3-noi.aces3.ai`, signing region `ap-south-noi-1`.
Object-storage SDK	`AWSSDK.S3` is the client library used against the Ace Cloud endpoint. Its package name does not mean the bucket is hosted by Amazon.

Component	Actual backend relationship
Background processing	Hosted PDF/report, email and webhook workers using durable regional database records and private stored objects.
External analytics	Secured Python bridge APIs; Python runtime/model deployment is managed separately.

2. Deployment workflow

Phase 1 — Prepare: source, build, test and package

Step 1. Retrieve approved source and identify the release

1. Retrieve frontend and backend source from the approved main/release branch. Resolve local changes before updating a release checkout; do not overwrite unrelated work.
2. Record the backend commit, frontend commit, mobile compatibility requirements and external Python dependencies in the release manifest.
3. Allocate a unique release ID and retain the last known-good full application artifact, configuration backup, database restore reference and release notes.
4. Identify changed APIs, database prerequisites, authentication behavior, worker behavior and any required app/frontend coordination before building.

Fetching source does not update IIS, apply SQL migrations or configure production secrets. Those are separate controlled steps.

Step 2. Build the components using their actual tooling

For the backend, run from the repository root:

```
dotnet restore SignalTracker.csproj
dotnet build SignalTracker.csproj -c Release --no-restore
dotnet build CallAnalyzerRegression/CallAnalyzerRegression.csproj -c Release
```

The Visual Studio folder publish profile uses Release and writes to `bin\Release\net8.0\publish`. It is a local filesystem publish profile, not a complete staging/production deployment pipeline. Its settings do not guarantee that a subsequent manual copy preserves the live server configuration.

Have the frontend owner build the React assets using the command and dependency lockfile from that frontend repository. Do not assume a particular Node version or package manager without that source. Build Python deliverables separately where the release changes the secured bridge contract or computed-result schema.

Step 3. Test, review dependencies and create immutable artifacts

The backend regression project is a console runner. Execute its supported modes explicitly; `dotnet test` alone does not execute this console suite.

```
dotnet run --project CallAnalyzerRegression -c Release --no-build -- --rf-mobile-upload
dotnet run --project CallAnalyzerRegression -c Release --no-build -- --rf-log-integration
```

```
dotnet run --project CallAnalyzerRegression -c Release --no-build -- --session-continuity
dotnet run --project CallAnalyzerRegression -c Release --no-build -- --security-baseline docs/security/endpoint-inventory.csv
```

Select diagnostic/CSV/cache/L3 regression modes when those areas change. Database-backed modes require a designated disposable/staging environment; inspect their fixtures before running them. The RF regressions cover authorization, automatic enrollment/renewal, ownership, report generation and email behavior with test stores/loopback SMTP. They do not establish real-phone acceptance or inbox delivery.

Run the existing baseline helper and inspect its reports:

```
.\scripts\vapt-baseline.ps1
dotnet list SignalTracker.csproj package --vulnerable --include-transitive
```

A successful command exit code does not mean that vulnerability reports are empty. Previous builds reported ImageSharp advisories; recheck the exact release dependency set and record remediation or an approved exception. SAST, secret scanning, authenticated dynamic testing and independent VAPT must be configured/run separately where required; the baseline script is not a security certification.

Create the backend release ZIP:

```
.\scripts\Build-RfLogRelease.ps1 -ReleaseName stracer-20261030-01
```

The script publishes Release without restore, removes published root `appsettings*.json`, bundles deployment/configuration helpers, RF SQL and integration documentation, creates `artifacts\stracer-20261030-01.zip`, and prints its SHA-256 hash. Restore must already have completed. It rejects an existing release folder/ZIP; use a new ID instead of replacing an accepted artifact.

Inspect the package for local uploads, credentials, logs, development settings, keys and unintended files. Record hashes for both frontend and backend artifacts. Promote the exact tested backend ZIP. For byte-identical frontend promotion, use an approved runtime API-URL configuration mechanism; if the frontend compiles environment URLs into its build, record separate environment artifact hashes rather than claiming the files are identical.

Phase 2 — Validate: staging deployment and acceptance

Step 4. Deploy the frontend to staging

Deploy the approved frontend build to its actual staging host. Configure the staging backend URL, HTTPS certificate, SPA routing, cache behavior and security headers. Validate that browser requests use the staging API and that the origin is explicitly allowed by staging CORS settings.

Use separate staging credentials, test users, MySQL schemas/instances, Redis namespace/service, object-storage bucket and approved email recipients. The RF object-key paths do not automatically contain a staging/production namespace, so sharing the production bucket without a reviewed isolation design is inappropriate.

Before starting workers against cloned data, sanitize company email profiles and report-recipient queues. Alternatively keep `RfLogIntegration:ReportEmailsEnabled=false` until test-only destinations have been verified. Do not allow a staging deployment to email production clients accidentally.

Step 5. Prepare and deploy the backend to staging

Verify Windows/IIS prerequisites: .NET 8 Hosting Bundle, existing site/application pool, HTTPS binding, app-pool identity, filesystem permissions, network access and an explicit Production environment for a production-like staging process. Use staging values even when testing production-mode behavior.

The deployment helper is an update tool for an existing configured IIS application. It expects a live `appsettings.json`, an existing pool and separate non-overlapping package/live directories. First-install IIS provisioning is a separate task.

From the extracted release folder on the server, use the actual staging paths and pool name:

```
.\scripts\Deploy-RfLogIntegration.ps1 -PackagePath 'C:\Releases\stracer-20261030-01\backend' -BackendPath
'C:\Sites\STracer-Staging' -AppPoolName 'ACTUAL_STAGING_POOL'
```

The helper validates paths, creates a restricted configuration backup, stops/waits for the pool, copies application files while preserving runtime settings/data, updates hosting arguments, grants required folder access, imports IIS environment variables into its CLI subprocesses, runs RF migrations and the Ace Cloud storage check, then starts the pool after successful checks.

Normal application updates must **not** use `-ConfigureStorage`. That optional switch invokes initial/intentional integration configuration and regenerates reporting/webhook keys. Use it only when that change is planned and coordinated.

Step 6. Validate complete staging workflows

Validate portal email/password login, logout, password/reset behavior, roles and company ownership, active/inactive licenses, India/Taiwan routing, reports, maps, imports, downloads and relevant Python bridge operations. Test overlapping user/session IDs in different regional databases; the trusted identity/credential determines scope, not a caller-provided region override.

Verify the real browser's cookies, allowed origins, forwarded HTTPS scheme, CORS, CSRF and protected WebSocket connections. CORS is a browser policy, not authentication. The custom portal CSRF middleware compares `X-CSRF-TOKEN` with `st.csrf` when configured and when the request is authenticated. Verify effective browser cookie readability because global cookie policy requests HttpOnly while the CSRF cookie creation sets HttpOnly false; source intent alone does not prove the frontend can read that cookie.

For the default RF mobile flow, test: pending enrollment, administrator recognition/approval, automatic token retrieval, lost-response recovery, concurrent renewal conflict handling, bearer ZIP upload, revocation, expiry, PDF generation and the intended company's email. The optional trusted-login endpoint supports authorization without manual approval using a pinned RSA public key and a short-lived installation-bound assertion. It is disabled by default and has local regression coverage; external login verification/signing, app integration and live acceptance remain outstanding. Do not mark that requirement accepted by hiding the pending screen or signing the current anonymous mobile signup result without ownership verification.

Check actual dependency access and background work, not just page rendering. Test large/malformed ZIPs, missing map images, failed S3 transfers, database failure, SMTP failure, process restart and worker recovery. Missing source screenshots cannot be created by authentication or email configuration.

Step 7. Apply the staging decision gate

Approve promotion only when the exact artifact passes the agreed acceptance criteria and all blocking defects are resolved or explicitly accepted by the responsible owner. Record actual results, timestamps and release hashes.

If validation fails, inspect logs and dependency checks, correct code/configuration, allocate a new build identity if bytes change, rebuild and repeat staging tests. A configuration-only change must still be versioned in the deployment evidence. Do not label a pending enrollment, successful negative-auth test or SMTP acceptance as completed end-to-end acceptance.

Phase 3 — Release: controlled production rollout

Step 8. Coordinate frontend, backend and app compatibility

Promote the validated release under the same manifest/release ID. The backend helper does not deploy the frontend, Android APK or Python service; coordinate their owners and compatibility separately. This is coordinated promotion, not an implemented atomic or zero-downtime multi-component release.

Confirm production API URLs, allowed origins, TLS bindings, database identities, Ace Cloud settings, SMTP regions, worker toggles and persistent key storage. Frontend/mobile packages must not contain S3 keys, SMTP passwords, database credentials or a common installation bearer token.

Step 9. Back up and perform the production deployment

Before stopping IIS, retain the complete previous backend/frontend artifacts, current `web.config` and `appsettings*.json`, persistent key ring, required upload data, and recoverable database backups for both regions. The helper's automatic backup contains configuration only; it does not back up all previous binaries, databases, the external key ring or stored objects.

Preserve these live paths during updates:

```
web.config
appsettings*.json
App_Data
UploadedExcels
logs
```

Also preserve any separately configured Data Protection key directory. Copying a fresh Visual Studio publish folder over these files can remove RF/SMTP environment values and recreate the observed configuration 503 problem. A Git pull or build does not restore server-only settings.

From the extracted package folder on the Windows backend server:

```
.\scripts\Deploy-RfLogIntegration.ps1 -PackagePath 'C:\Releases\stracer-20261030-01\backend' -BackendPath
'C:\inetpub\wwwroot\Backend' -AppPoolName 'ACTUAL_BACKEND_POOL'
```

Use the real pool name from IIS Manager. `PackagePath` is the inner directory containing `SignalTracker.dll`. Schedule a maintenance window: this helper stops the pool during copying and checks. A failed migration or storage check leaves the pool stopped/may leave a partially updated release; inspect the error and recovery plan before reopening service.

The helper applies the embedded RF migration set to both regions. It sets AlwaysRunning and disables idle timeout, but IIS preload/Application Initialization must still be configured and verified for automatic startup after server restarts. Open the site once after deployment where needed. Neither process start nor an IIS recycle proves the workers can complete jobs.

Step 10. Run production smoke checks and observe behavior

Use `X-Public-Api-Key` with the actual configured `Security:PublicApiKey` for `/healthz/live` and `/healthz/ready`. Keep credentials out of command history, recorded output and screenshots. The following uses an operator-side environment variable name; it is not a new backend configuration key:

```
$headers = @{ 'X-Public-Api-Key' = $env:SIGNALTRACKER_HEALTH_KEY }
Invoke-RestMethod 'https://s-traccceer.vinfocom.co.in/healthz/live' -Headers $headers
Invoke-RestMethod 'https://s-traccceer.vinfocom.co.in/healthz/ready' -Headers $headers
```

Liveness verifies the process can respond. Readiness checks the default-context database and Taiwan database and reports Redis status, but Redis status does not determine the current overall readiness HTTP result. The probes do not test S3, SMTP, webhook receivers, Python models, a complete portal session or the mobile upload/report workflow.

Complete designated portal login/scoped reads and an approved app/API workflow. Interpret negative tests correctly: missing enrollment secret 400, unknown enrollment 404 and invalid bearer 401 are expected denials, not successful upload evidence. A configuration 503, database 503 or migration failure is a blocking defect.

Monitor database latency/connections, process restarts, HTTP error rates/latency, memory/CPU during report generation, pending/failed report jobs, delivery attempts, storage failures and webhook responses for the agreed observation window. Assign numerical thresholds and an owner before rollout; the repository does not define a production SLO or fixed acceptance window.

Step 11. Decide whether production is healthy

If all gates pass, record release notes, source/artifact identities, configuration version, schema results, smoke evidence, remaining accepted risks, monitoring observations and handover ownership. Retain the previous known-good artifact and backups through the agreed recovery period.

If a gate fails, stop further promotion, preserve logs/evidence and determine whether the previous application is compatible with the current schema. Restore a compatible complete application/configuration set, or apply a reviewed forward fix. Do not drop new tables or restore an old database over new transactions as an automatic rollback step. Repeat auth, regional, storage, worker and delivery checks after recovery.

3. Deployment principles specific to STracer

1. Build once and identify each artifact. One release manifest links frontend/backend/app compatibility; each component retains its own checksum.
2. Preserve server configuration and persistent data. Published binaries and runtime secrets have different lifecycles.
3. Use the deployed IIS application's effective configuration for CLI operations. User Secrets on a developer computer do not configure production.
4. Review database compatibility for both regions. RF migration success in one database is not success in the other.
5. Keep staging isolated. Real company email profiles and queued jobs require particular care before starting staging workers.
6. Preserve Data Protection keys and their decryption capability across restarts, identity changes, rollback and multiple instances.
7. Validate identity and ownership independently of transport encryption. TLS, company code, phone number and CORS are not substitutes for authentication.

8. Treat long-term installation secrets, short-term bearer tokens, portal cookies, service API keys, S3 keys and SMTP passwords as separate credential classes.
9. Inspect dependency/security findings and effective infrastructure settings. Documentation is not proof of encryption-at-rest or security certification.
10. Retain a complete known-good release and an exercised recovery procedure. The current deployment script does not automatically roll back a failure.

4. Deployment targets and configuration ownership

Target	Configuration and verification
Frontend host	Approved React asset host; correct production API URL, HTTPS, SPA routing and caching. Frontend build/hosting is outside the backend helper.
Backend IIS	.NET 8 Hosting Bundle; site binding; correct pool identity; `ASPNETCORE_ENVIRONMENT=Production`; writable runtime/temp directories; readable protected configuration and key ring.
MySQL IN/TW	Separate connection-string keys and explicitly selected region. Confirm schemas, restore capability, TLS policy, account privileges and connection limits. Do not assume TLS from the existence of a connection string.
Redis	Configure connectivity/authentication and TLS if required by the environment. Verify real cache behavior and regional database-backed login-lock behavior during outages.
Ace Cloud storage	`dcra-bucket`, actual HTTPS endpoint/signing region, private-object access and scoped credentials. The check command also needs delete permission on its temporary check prefix.
SMTP	Configure IN and TW separately. Explicit invalid settings fail; no cross-region SMTP fallback is intended. Confirm the sender account, app password, TLS and company recipient profile.
Webhooks /Python	Separate credentials, HTTPS, scoped access, receiver validation and version-compatible request/results schemas. A model runtime is not deployed by this IIS helper.

Use server `web.config` environment entries or an approved production secret provider for deployment values. `appsettings.Production.example.json` is only an example and is not automatically loaded. Never deploy literal placeholder key strings as actual credentials.

Important key groups are `ConnectionStrings__MySqlConnection`, `ConnectionStrings__MySqlConnection2`, `ConnectionStrings__Redis`, `RfLogIntegration__*`, `PythonBridge__ApiKey`, `Security__PublicApiKey`, `Security__AllowedOrigins__*`, `Security__RequireCsrfHeader`, trusted proxy settings, SMS provider keys, and `DataProtection__KeysPath` or `DATAPROTECTION_KEYS_PATH`.

The example environment syntax uses double underscores for nested keys. In Development the application explicitly reloads its user-secrets JSON after default configuration setup, so do not assume default precedence without checking the effective Development configuration.

5. Encryption, decryption, hashing and salt

5.1 Terms and actual coverage

Encryption protects confidentiality and can be reversed with the correct key. A password hash is one-way verification data; it is not decrypted during modern password login. A salt makes password-derived values distinct; it is different from an encryption key or IV. An HMAC authenticates data with a secret key; it does not hide the data. Base64 and hexadecimal are encodings, not encryption.

Data / mechanism	Current implementation	Meaning and limitation
New portal passwords	BCrypt, work factor 12	Salted one-way password hashing; no password decryption for modern bcrypt records.
Password salt	Generated automatically by BCrypt for each hash	No application-wide fixed salt is passed to `HashPassword`. The stored bcrypt string contains the salt, cost and hash needed for verification.
Portal auth cookie	ASP.NET Core Data Protection	Encrypted/authenticated ticket; preserving the key ring is essential. This does not encrypt all database records or files.
Password reset link	Time-limited Data Protection purpose `SignalTracker.PasswordReset.v2`	Protected payload with a 15-minute lifetime.
RF access token database storage	SHA-256 digest of the full bearer token	No raw token is recovered by decrypting the database value. High-entropy secrets, rather than human passwords, make this hash use appropriate.
Enrollment installation secret storage	SHA-256 digest	The app generates 32 random bytes, represented as 64 lowercase hex characters; the server stores its proof digest.
Automatically renewed RF token	HMAC-SHA256 derivation from the installation secret and renewal context, then SHA-256 digest storage	Deterministic recovery for the current renewal, with atomic previous-renewal comparison and revocation checks. It is an opaque bearer, not a JWT.
Optional manual RF credential	32 bytes from `RandomNumberGenerator.GetBytes`, hex encoded	Random token secret, scoped to user/company/region and stored as a digest.
RF webhook signature	HMAC-SHA256 over timestamp + `.` + JSON body	Integrity/authenticity; receiver must check timestamp freshness and deduplicate event IDs. The signature itself does not encrypt event contents.
API-to-client and API-to-Ace Cloud transfers	HTTPS	Certificate validation and TLS protect transport. Actual protocol/cipher policy depends on hosting/OS/provider configuration.
RF SMTP transfer	Required STARTTLS on TLS-enabled non-465 ports; TLS-on-connect on 465	Transport encryption to SMTP provider; not end-to-end mailbox/PDF encryption. `EnableSsl=false` remains a possible configuration and must not be used unintentionally.
Database files, Redis persistence, S3 stored objects and backups	Infrastructure/provider controls	Application code does not establish encryption-at-rest for these components. Verify and record the actual policy.

5.2 Portal password hashing and salt handling

`Security/PasswordSecurity.cs` creates new hashes using `BCrypt.Net.BCrypt.HashPassword(password, workFactor: 12)` and verifies them with BCrypt. The library generates a new salt internally for each hash. There is no separate global bcrypt salt to enter into `appsettings.json`, share with the app developer or retrieve during login.

Legacy compatibility exists for earlier SHA-256 representations, plaintext/Base64 forms and legacy AES-decryptable values when the login code permits fallback. Successful supported legacy logins can upgrade non-bcrypt records. This compatibility is not evidence that every database password has already been migrated; inventory and retire legacy representations through a controlled plan. Client-side SHA-256 is not a replacement for HTTPS or backend password hashing.

5.3 Legacy AES encryption/decryption: exact helper behavior

`Helpers/AESEncrytDecry.cs` contains a separate legacy AES helper. It derives a key through `.NET PasswordDeriveBytes(password, saltBytes, hashName, iterationCount)`, encrypts UTF-8 payloads in AES-CBC mode and Base64-encodes the ciphertext. Decryption reverses the Base64/AES operation using the same derivation inputs and IV. The helper relies on the AES provider's default padding rather than explicitly selecting a padding mode.

The source defaults are a 256-bit key size, SHA-1 for the legacy derivation setting and an iteration count of 2. The salt and IV/vector are configured strings converted to ASCII bytes; they are not automatically generated per encryption operation. Environment overrides include `SIGNALTRACKER_AES_SALT`, `SIGNALTRACKER_AES_VECTOR`, `SIGNALTRACKER_AES_PASSWORD`, `SIGNALTRACKER_AES_HASH`, `SIGNALTRACKER_AES_ITERATIONS` and `SIGNALTRACKER_AES_KEY_SIZE`. The class also exposes legacy AES key/IV settings. Do not publish actual configured secret values in a deployment document.

These are **legacy compatibility mechanisms, not a modern authenticated-encryption design for new sensitive data**. The low derivation iteration default, reusable vector/default material and unauthenticated CBC behavior require a dedicated compatibility/security review before reuse. Do not claim that this helper supplies authenticated encryption or encrypts the entire system. Changing its configuration without a data migration can make historical ciphertext unreadable.

5.4 Data Protection cookies and key management

`Configuration/SecurityServiceExtensions.cs` configures `PersistKeysToFileSystem(...)` and application name `SignalTracker`. `HostingConfiguration` selects the configured `DataProtection:KeysPath`, then the alternate environment path, with environment-specific defaults. The production example selects `App_Data/DataProtectionKeys`; verify the actual effective directory and app-pool permissions instead of assuming that path.

Microsoft documents the framework's default payload protection as AES-256-CBC with HMACSHA256. The repository does not explicitly override the payload algorithms, so verify the deployed key descriptors and any machine policy before claiming the exact active algorithm. Microsoft Data Protection defaults.

Key-ring encryption at rest is not explicitly configured in this repository. Explicit filesystem persistence disables automatic at-rest key protection unless an explicit protector is added. Restricted ACLs are necessary but are not encryption. Review DPAPI for an appropriate single-machine/identity deployment, or an approved certificate/key-management arrangement for shared/multi-instance use. Do not claim these options are implemented merely because this document names them. Microsoft key-encryption guidance.

Preserve retired keys needed to read existing protected data. Back up keys securely and verify recovery with the correct identity/certificate. A multi-instance deployment needs a shared, appropriately protected key ring and compatible application name; deleting keys or changing identity without recovery can invalidate cookies/reset tokens.

5.5 Secrets, database/object encryption and transport boundaries

User Secrets stores development values outside the repository but does not encrypt its JSON. Environment variables and IIS environment entries also must not be described as an encrypted vault. Protect server access, configuration ACLs, backups and operator output; integrate an approved secret provider if stronger secret-at-

rest controls are required. Microsoft app-secrets guidance.

The Ace Cloud adapter uses an HTTPS custom endpoint and S3-compatible signed requests/short-lived URLs. ZIP signatures and size/ETag checks verify expected transfer properties; a ZIP magic signature is not a digital signature, and an ETag is not universally a cryptographic file checksum. The adapter does not explicitly request server-side object encryption/KMS in its PutObject calls. Ace Cloud bucket encryption may be configured by the provider, but it must be independently verified. Private bucket access and encrypted storage are separate properties.

Similarly, no blanket field-level encryption of all MySQL records or explicit forced Redis TLS is established by the inspected code. Verify connection TLS/certificate policies, database/volume encryption, Redis persistence encryption and backup encryption in the deployed environment. PDFs are not password-encrypted by this report pipeline; email attachments and locally downloaded copies remain sensitive after transport completes.

IIS handles the public certificate/HTTPS binding and the operating system negotiates TLS. HSTS and HTTPS redirection are enabled in production code; trusted forwarded-header settings must correctly identify HTTPS at a reverse proxy. Do not claim a minimum TLS version, particular cipher suite or encrypted IIS-to-proxy hop without checking that infrastructure configuration.

6. Database migrations and regional isolation

RF migration	Role
003	RF upload/event/report job schema.
004	RF report-recipient and email-delivery persistence.
005	Adds the company-default recipient marker; the helper checks for the column before applying.
006	Installation upload credentials, scope, hashes, expiry and revocation.
007	Pending/approved device enrollment, proof hashes, approval expiry and renewal coordination.

The RF migration command applies/checks this embedded set in both configured regions; it is not a general runner for every file in `Database/Migrations`. Review other release prerequisites separately. Production startup does not execute the Development-only startup-maintenance block, but service-specific ensure-table/schema behavior still requires review; do not assume every runtime operation is DDL-free.

```
dotnet SignalTracker.dll --rf-log-db-check --environment Production
dotnet SignalTracker.dll --rf-log-db-migrate --environment Production
dotnet SignalTracker.dll --rf-log-storage-check --environment Production
```

Run from the deployed application folder with the same effective configuration as IIS. Plain CLI execution does not import web.config environment entries automatically; the deployment/repair helpers do that explicitly. `db-check` is a schema/connectivity check; `db-migrate` changes schema; `storage-check` writes and removes disposable Ace Cloud objects. A local Development command may still target remote/production databases if its connection strings point there.

Signed portal identities and RF credentials bind company, user and region. Reject or ignore untrusted region/company overrides according to the endpoint's design. Do not repair orphaned user/company mappings by guessing a tenant from similar names or numeric IDs; obtain an authoritative mapping first.

7. Authentication and automatic report behavior

Portal email/password login issues protected `st.auth` and server-session `st.session` cookies. Role/company/license and regional login-lock checks control continued access. Current `SessionSecurity` defaults `KeepSignedInUntilLogout` to true and renews long-lived tickets; older short-session documentation must not be assumed to describe this release. Record the configured session policy and validate logout, password/role/company/license changes and takeover handling. The regional database login lock is authoritative in the current implementation; Redis configuration alone does not prove session validity.

RF automatic enrollment currently uses a client-generated 32-byte secret and pending administrator approval. Pending requests expire after 48 hours; access tokens normally last 30 days, with a one-day suggested renewal lead time; approval is capped at 365 days. Renewal uses `previousRenewalId` to prevent a stale competing rotation from overwriting a newer one. Revoked or invalid ownership cannot be revived through renewal. See `docs/rf-mobile-auto-enrollment.md` for the exact contract.

The desired no-approval-popup mobile experience requires the external login backend to establish trusted ownership/device verification and issue the signed assertion accepted by the optional RF trusted-login endpoint. Its configuration is `RfLogIntegration:TrustedMobileLogin`; it uses RSA-SHA256 signatures, a pinned public key scoped to one region and explicit company IDs, a maximum 120-second assertion lifetime, and enrollment UUID/secret-hash binding. The login server retains the private key. Existing anonymous signup, numeric IDs and Android ID alone do not establish this trust. The integration is disabled by default; the currently deployed approval requirement must not be represented as already replaced until login-server/app rollout and live testing are complete.

After a valid upload, the backend verifies user/session/project ownership, writes/verifies a private ready object and records completion. Workers claim regional jobs with leases, generate PDF using the existing ZIP report engine, save the PDF privately and queue email. Default delivery is to the company's registered `tbl_company.email`, plus eligible configured recipients. It is not automatically whichever portal email happens to be logged in, nor the SMTP sender account.

The app's multipart upload uses `file`, stable `uploadId` and `sessionId`; optional project IDs must be positive or omitted. Retry unchanged ZIPs under the same identity and UUID. Reusing an ID with changed metadata can produce 409. Bearer-only native upload does not require portal credentials/CSRF; cookie-authenticated portal requests retain existing CSRF rules.

Current defaults include a 100 MiB HTTP body cap, configurable up to 500 MiB, with IIS/proxy limits and multipart overhead also applying. The separate default RF ZIP limit is 1 GiB and does not override the smaller single-call HTTP cap. Automatic report safeguards include 2 GiB total expanded data, a 256 MiB per-entry cap, 5,000 archive entries and 250,000 CSV lines. The default email-attachment cap is 20 MiB. Confirm effective overrides and realistic performance before increasing limits.

Idle report/email workers normally poll about every 15 seconds; processing time depends on workload. Reports are not guaranteed to be emailed at the instant a drive ends. An upload 200 confirms acceptance, an SMTP acceptance confirms handoff, and neither proves inbox receipt. Retry/fencing/deduplication reduce duplicate work; uncertain SMTP handoff followed by a failed database update must still be investigated rather than claiming exactly-once email delivery.

8. Operational failure diagnosis

Symptom	Investigation / action
RF 503: storage integration not configured	Check effective Enabled, endpoint, bucket, signing region, access/secret keys and numeric limits. Confirm the correct IIS site/folder. Preserve configuration on deployment; do not repeatedly rotate keys as a substitute for fixing overwrite behavior.

Symptom	Investigation / action
RF/enrollment database 503	Verify regional credentials/connectivity, all required tables including 006/007, and the actual IIS database targets. A developer machine's successful check does not prove the server uses the same values.
RF 401	Missing, malformed, expired or revoked bearer, or wrong existing integration credentials. Do not add S3/SMTP keys to the phone or make the upload anonymous.
Enrollment pending/token 403	Expected before trusted approval on the default route; also check inactive/mismatched ownership. The new trusted route requires enabled issuer/public-key configuration and an installation-bound verified-login assertion. Older apps using the default route still receive pending status.
Upload 403	Session/user/company/project mismatch; correct source ownership, not merely the displayed region or phone number.
Upload 400/413	Validate UUID/form/ZIP signature and the lowest application/IIS/proxy size limit, including multipart overhead.
Report accepted but not generated	Check worker startup, regional job status, archive/row limits, supported NetworkLog CSV, leases and error state.
Report lacks maps	Inspect screenshots actually contained in the source ZIP and recognized metric filenames. Do not insert unrelated session images or fabricate missing measurements.
PDF exists but email missing	Check company recipient profile/subscription, regional SMTP, TLS/app password, attachment size, delivery attempt state and spam/bounce results.
Login invalid after deployment	Check key-ring preservation/access and actual session/account/license/login-lock state before changing credentials.
Pool stopped after deployment failure	Read the failed gate and restricted backup location; choose a reviewed repair or compatible rollback, then repeat smoke tests.

Use `Repair-RfLogStorage.ps1` for a deliberate storage repair and `Configure-RfLogReportEmail.ps1` for regional SMTP configuration. These are not routine replacements for preserving live configuration. Restrict log access and retention. Audit debug/provider URL logging, especially SMS requests that can include API credentials and OTP content, before retaining or sharing logs.

9. Rollback, key recovery and operational ownership

Maintain a named release owner, backend operator, frontend owner, database owner, mobile integration owner and SMTP/storage contact. Record who may approve devices and who may rotate each credential type. Keep changes to company recipients and test-email permission explicit.

Rollback restores a complete compatible application artifact and appropriate configuration, not an arbitrary mixture of DLLs. The current helper copies files without pruning every obsolete binary, so evaluate stale-file compatibility during release review. Database rollback is a separate recovery decision; additive tables can often remain for an older compatible application, while destructive changes require a planned restore/forward-fix strategy.

Preserve in-flight uploads/job records and company delivery history. Reuse stable identifiers for retries. Recover the Data Protection key ring with its correct decryption identity/certificate; restoring encrypted files without their keys is not a successful recovery. Validate restored object/database consistency and SMTP/job behavior before reopening traffic.

10. Release acceptance checklist

- [ ] Approved source commits, release ID, component compatibility and frontend/backend artifact hashes recorded.
- [ ] Release build, relevant regressions, dependency review and configured security scans completed; blocking findings resolved or explicitly accepted.
- [ ] Frontend source/build/hosting verified by its owner; staging and production API URLs and origins correct.
- [ ] Staging integrations isolated; no unintended production recipients or queued client email.
- [ ] Full previous artifacts/configuration, both database restore references and persistent keys backed up and recoverable.
- [ ] IIS deployment preserves web.config, appsettings files, key directories, runtime data and uploads.
- [ ] RF migrations 003–007 checked in both actual deployment databases; other release schema prerequisites reviewed.
- [ ] Ace Cloud bucket/region/access and real storage round trip verified; no temporary check objects left behind.
- [ ] Effective password, cookie/session, CSRF, role/company/region and mobile credential controls tested.
- [ ] Data Protection key-at-rest, MySQL/Redis TLS, bucket/volume/backup encryption and log redaction verified or recorded as unresolved gaps.
- [ ] IN/TW SMTP configuration and approved recipient delivery verified with the intended PDF attachment.
- [ ] Mobile automatic-upload acceptance recorded; the approval-free mobile-authentication requirement is not incorrectly marked complete.
- [ ] Liveness/readiness, actual protected requests, workers and integration checks pass; negative tests are correctly interpreted.
- [ ] Monitoring thresholds, observation period, rollback owner, release notes and handover completed.

11. Source evidence and limitations

Primary repository references: `SignalTracker.csproj`, `Program.cs`, `Properties/PublishProfiles/FolderProfile.pubxml`, `web.config`, `scripts/Build-RfLogRelease.ps1`, `scripts/Deploy-RfLogIntegration.ps1`, `Configuration/SecurityServiceExtensions.cs`, `Configuration/HostingConfiguration.cs`, `Security/PasswordSecurity.cs`, `Security/PasswordResetTokens.cs`, `Security/SessionSecurity.cs`, `Helpers/AESEncrytDecry.cs`, `Services/RfMobileCredentialService.cs`, `Services/RfMobileEnrollmentService.cs`, `Services/RfLogDatabaseSetup.cs`, `Services/RfLogS3Storage.cs`, `Services/RfLogReportEmailWorker.cs` and `Controllers/HealthController.cs`.

This revision documents source behavior and prior observed integration boundaries. No deployment, credential rotation, database change, client email or encryption-code change is performed by preparing this document. Infrastructure encryption policies, complete frontend builds, the approval-free mobile integration and full production end-to-end acceptance require their own current evidence.